

Comparación de 3 Optimizadores

Modelo Depredador-Presa (Lotka-Volterra)

Aplicado a la optimización de una app de citas

Emmanuel Guerrero Piza – 20202020039

Kevin Andrés Forero Guaitero – 20181020120

UNIVERSIDAD DISTRITAL

May 13, 2026

Contenido

- 1 Definición del problema
- 2 Optimizadores
- 3 Optimizador 1: Differential Evolution
- 4 Optimizador 2: SGD
- 5 Optimizador 3: ANFIS
- 6 Comparaciones
- 7 Conclusiones

Modelo Lotka-Volterra para App de Citas

MODELO LOTKA-VOLTERRA (APP DE CITAS)

$$\begin{aligned}x_{\text{dot}} &= a*x - b*x*y && \text{(perfiles/parejas)} \\y_{\text{dot}} &= c*x*y - d*y && \text{(usuarios activos)}\end{aligned}$$

PARAMETROS:

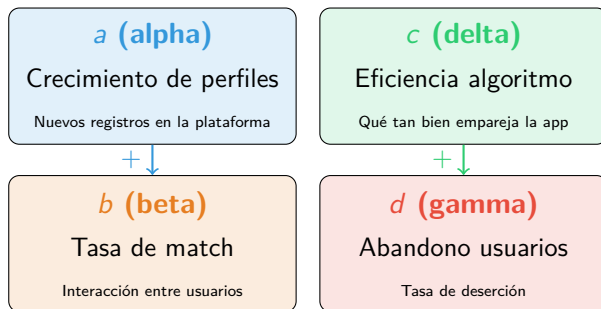
- a (alpha) = crecimiento de perfiles
- b (beta) = tasa de match
- c (delta) = eficiencia del algoritmo
- d (gamma) = abandono de usuarios

PUNTO DE EQUILIBRIO:

$$x^* = d/c \quad y^* = a/b$$

El sistema modela la interacción entre perfiles disponibles (x) y usuarios activos (y). El objetivo es encontrar parámetros que maximicen la rentabilidad manteniendo un equilibrio estable.

Parámetros del sistema



Ecuaciones del modelo

$$\dot{x} = a x - b x y$$

$$\dot{y} = c x y - d y$$

x
perfiles

y
usuarios activos

Punto de equilibrio

$$x^* = \frac{d}{c} \qquad y^* = \frac{a}{b}$$

El sistema converge a (x^*, y^*) independientemente de las condiciones iniciales.

Estado inicial (línea base)

Parámetros por defecto:

$$a = 0.3, b = 0.006, c = 0.018, d = 0.7$$

Condiciones iniciales:

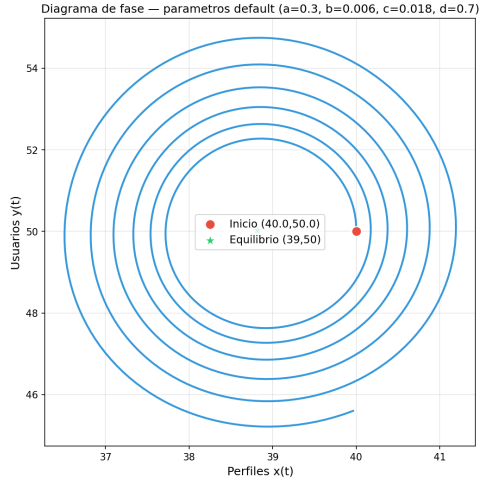
$$x_0 = 40, y_0 = 50$$

Métricas base:

- Costo: -27.95
- CV: 0.059 (estable)
- Retención: 49.7%
- Ganancia: $\$62.49/\text{mes}$

```
% Parámetros iniciales del sistema  
T = 0.1 # Tiempo de muestreo  
X0, Y0 = 40, 50 # Condiciones iniciales  
% Límites de búsqueda del optimizador  
ALPHA_BOUNDS = (0.2, 1.0)  
BETA_BOUNDS = (0.005, 0.04)  
DELTA_BOUNDS = (0.002, 0.04)  
GAMMA_BOUNDS = (0.1, 0.7)
```

Diagrama de fase — Estado inicial



DE Differential **E**volution

Búsqueda evolutiva global con población de individuos

SGD Stochastic **G**radient **D**escent

Gradiente descendente estocástico con momentum

NFIS Adaptive **N**euro-**F**uzzy

Inference **S**ystem

Sistema de inferencia neuro-difuso

Notación — Métricas de evaluación

CV Coefficient of Variation

$$CV = \frac{\sigma(y)}{\mu(y)} \text{ en estado estable}$$

Mide la estabilidad del sistema ($< 0.3 = \text{estable}$)

x^* Equilibrio de perfiles

$$x^* = \frac{d}{c}$$

y^* Equilibrio de usuarios

$$y^* = \frac{a}{b}$$

Resumen de métricas de comparación

Métrica	Mide	Mejor
Costo (J)	Función objetivo (penaliza desbalance + inestabilidad, recompensa ganancia/ret.)	más negativo
CV	Estabilidad del sistema	menor
Retención	e^{-d} , fracción de usuarios que no abandonan	mayor
Ganancia	Ingreso estimado en \$/mes	mayor
Tiempo	Segundos de ejecución	menor

Arquitectura de los optimizadores

Entradas y salidas del sistema de optimización

Entradas (inputs)

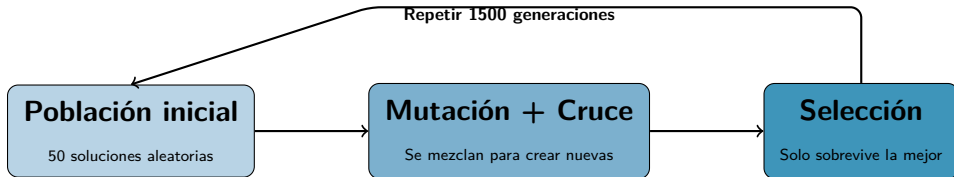
- Parámetros del modelo: a, b, c, d con cotas
- Condiciones iniciales: $x_0 = 40$, $y_0 = 50$
- Tiempo de muestreo: $T = 0.1$, $N = 800$ pasos
- Función de costo: $J(a, b, c, d)$

Salidas (outputs)

- Parámetros óptimos: a^*, b^*, c^*, d^*
- Punto de equilibrio: (x^*, y^*)
- Trayectorias: $x(t), y(t)$ simuladas
- Métricas: costo J , CV, retención, ganancia

Objetivo: encontrar (a^*, b^*, c^*, d^*) que minimicen J sujeto a las cotas de cada parámetro.

Algoritmo evolutivo — inspirado en la selección natural



Analogía

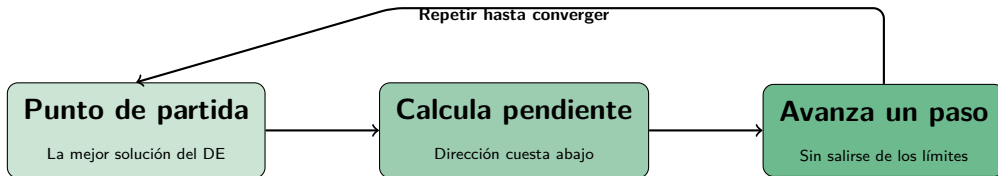
Como una competencia: generamos muchas versiones de los parámetros, las combinamos, y solo gana la que da mejor resultado en cada ronda.

Después del DE

Se aplica L-BFGS-B para refinar localmente la mejor solución encontrada.

L-BFGS-B — ¿Cómo funciona?

Optimizador local basado en gradiente — el refinador



¿Qué significa el nombre?

Limited memory: solo recuerda los últimos pasos

BFGS: método que aproxima la curvatura

Box: respeta límites (caja) en cada parámetro

Analogía

Como bajar una montaña con los ojos cerrados: sientes la pendiente con el pie (*gradiente*), das un paso en esa dirección, y repites hasta llegar al valle. Las *cajas* son cercos que impiden salir del camino permitido.

L-BFGS-B vs SGD — Diferencias clave

Dos formas de bajar la montaña

L-BFGS-B

- Usa el **gradiente exacto** (diferencias finitas)
- Estima la **curvatura** del terreno (Hessiana aproximada)
- Un solo camino, muy pulido
- Pasos más inteligentes (tamaño adaptativo)

Analogía: Caminas con un mapa topográfico — sabes la pendiente y la forma del terreno.

SGD

- Usa un **gradiente aproximado** ruidoso
- Ignora la curvatura (solo dirección)
- Múltiples reinicios desde distintos puntos
- Momentum para no quedarse atascado

Analogía: Lanzas 50 personas desde distintos puntos de la montaña, cada una baja a ciegas sintiendo el piso.

DE — Differential Evolution

Estrategia:

- Búsqueda global evolutiva sin gradientes
- Población de 50 individuos, 1500 generaciones
- Refinamiento local con L-BFGS-B

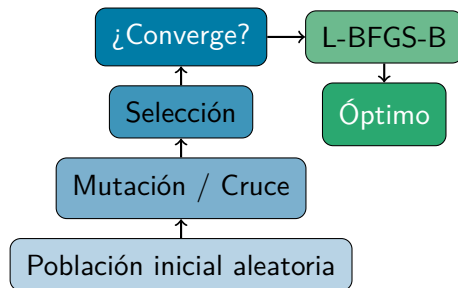
Parámetros óptimos:

$$a = 0.279, b = 0.005, c = 0.018, d = 0.700$$

$$x^* = 39.1, y^* = 55.7$$

Métricas:

- Costo: **-27.95** (mejor)
- CV: 0.103 (estable)
- Ganancia: **\$70.60/mes** (mejor)
- Tiempo: 13.0s



DE — Código: optimización global

Núcleo del optimizador: búsqueda evolutiva con `differential_evolution` seguida de refinamiento local L-BFGS-B.

```
class DifferentialEvolutionOptimizer:
    def optimize(self, n_steps=800, x0=X0, y0=Y0):
        bounds = [ALPHA_BOUNDS, BETA_BOUNDS,
                  DELTA_BOUNDS, GAMMA_BOUNDS]

        # Fase 1: búsqueda global evolutiva
        res = differential_evolution(
            lambda p: cost_function(p, n_steps, x0, y0),
            bounds,
            strategy='best1bin',      # mutación diferencial
            maxiter=1500,             # generaciones
            popsize=50,               # individuos por generación
            tol=1e-8, seed=42,
        )

        # Fase 2: refinamiento local
        res2 = minimize(
            lambda p: cost_function(p, n_steps, x0, y0),
            res.x, bounds=bounds, method='L-BFGS-B')

        xopt = res2.x if res2.fun < res.fun else res.x
        self.a, self.b, self.c, self.d = xopt
        self.x_sim, self.y_sim = simulate(*xopt, n_steps, x0, y0)
        return self
```

`differential_evolution` explora el espacio de búsqueda de forma paralela con una población de 50 individuos. L-BFGS-B refina la mejor solución encontrada.

SGD — Stochastic Gradient Descent

Estrategia:

- Gradiente aproximado por diferencias finitas
- Momentum (0.85) + clipping de gradiente
- Grid de > 50 puntos de inicio + reinicios aleatorios
- Learning rate con decaimiento ($\times 0.99$)

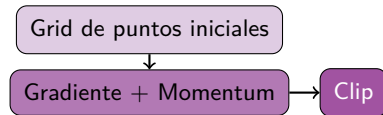
Parámetros óptimos:

$$a = 0.300, b = 0.006, c = 0.018, d = 0.700$$

$$x^* = 38.9, y^* = 50.0$$

Métricas:

- Costo: -25.34
- CV: **0.041** (más estable)
- Ganancia: \$61.72/mes
- Tiempo: 19.9s



SGD — Código: gradiente descendente

Núcleo del optimizador: gradiente aproximado por diferencias finitas con momentum y grid de reinicios.

Gradiente por diferencias finitas:

```
def _finite_grad(self, p, n_steps, x0, y0, eps=1e-4):
    base = cost_function(p, n_steps, x0, y0)
    grad = np.zeros(4)
    for i in range(4):          # perturba cada parámetro
        p_up = p.copy()
        p_up[i] += eps
        p_up = self._clip(p_up)
        grad[i] = (cost_function(p_up, n_steps, x0, y0) - base) / eps
    # clipping para evitar explosiones cerca de penalizaciones
    norm = np.linalg.norm(grad)
    if norm > 1000:
        grad = grad / norm * 1000
    return grad, base
```

Bucle de descenso con momentum:

```
def _restart_from(self, p0, n_steps, x0, y0, max_iter=150, lr=0.005):
    p = p0.copy(); v = np.zeros(4)          # momentum inicial
    best_p = p.copy()
    best_c = cost_function(p, n_steps, x0, y0)
    for step in range(max_iter):
        grad, _ = self._finite_grad(p, n_steps, x0, y0)
        v = 0.85 * v + lr * grad            # momentum
        p = self._clip(p - v)                # actualización
        lr *= 0.99                           # decaimiento
    ...
```

ANFIS — Neuro-Fuzzy Optimizer

Estrategia:

- 3 funciones de membresía Gaussianas por parámetro (Low, Medium, High)
- Red neuronal feedforward ($12 \rightarrow 16 \rightarrow 4$) ajusta centros y anchos
- Inferencia difusa: reglas heurísticas basadas en el costo
- Entrenamiento: muestreo $\rightarrow$ evaluar $\rightarrow$ retropropagar

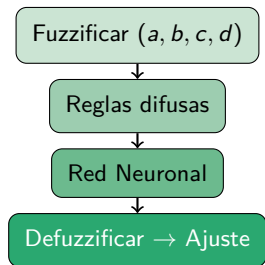
Parámetros óptimos:

$$a = 0.883, b = 0.013, c = 0.004, d = 0.269$$

$$x^* = 63.8, y^* = 66.9$$

Métricas:

- Costo: -12.11
- CV: 0.338
- Retención: **76.4%** (mejor)
- Tiempo: **0.5s** (más rápido)



ANFIS — Código: sistema neuro-difuso

Núcleo del optimizador: fuzzificación de parámetros + red neuronal para ajuste fino.

Fuzzificación con Gaussianas:

```
def _fuzzify(self, params):
    a, b, c, d = params
    x = np.array([a, b, c, d])
    phi = []
    for i in range(4):
        # para cada parámetro
        for j in range(self.n_mf):
            # Low, Medium, High
            val = exp(-(x[i] - mu[i,j])**2) / (2 * sigma[i,j]**2))
            phi.append(val)
            # grado de membresía
    return np.array(phi)
    # vector de 12 características
```

Red neuronal feedforward (12→16→4):

```
def _nn_predict(self, phi):
    h = np.tanh(self.W1.T @ phi + self.b1)
    out = self.W2.T @ h + self.b2
    return np.tanh(out) * 0.1
    # capa oculta
    # capa de salida
    # ajuste acotado

def _nn_train(self, phi, target, lr=0.01):
    pred = self._nn_predict(phi)
    error = pred - target
    d_W2 = np.outer(h, error * (1 - tanh(out)**2) * 0.1)
    d_W1 = np.outer(phi, self.W2 @ d_out * (1 - h**2))
    self.W2 -= lr * d_W2
    self.W1 -= lr * d_W1
    # error de ajuste
    # retropropagación
```

Configuración de entrada — 3 optimizadores

Parámetros comunes del sistema Lotka-Volterra:

Condiciones iniciales

$$x_0 = 40, \quad y_0 = 50$$

Tiempo de muestreo: $T = 0.1$

Función objetivo

Costo = Ganancia - Penalizaciones

$$J = -(0.6 G - 0.3 CV - 0.1 R^{-1})$$

DE

pobl=50, gen=1500

L-BFGS-B

SGD

lr=0.005, mom=0.85

grid > 50 reinicios

ANFIS

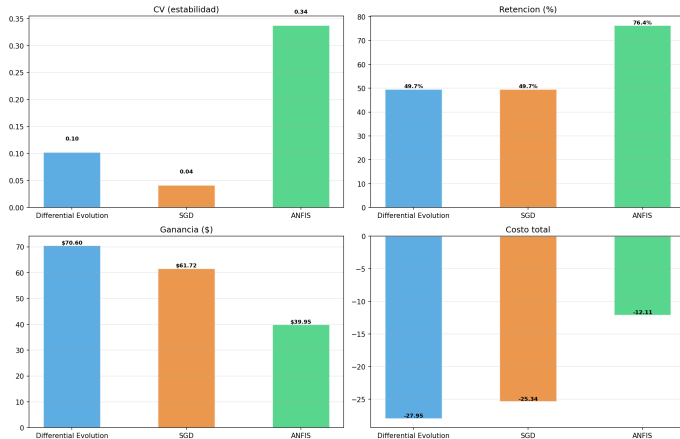
3 MF Gaussianas

red 12→16→4

Límites de búsqueda: $\alpha \in [0.2, 1.0]$, $\beta \in [0.005, 0.04]$, $\delta \in [0.002, 0.04]$, $\gamma \in [0.1, 0.7]$

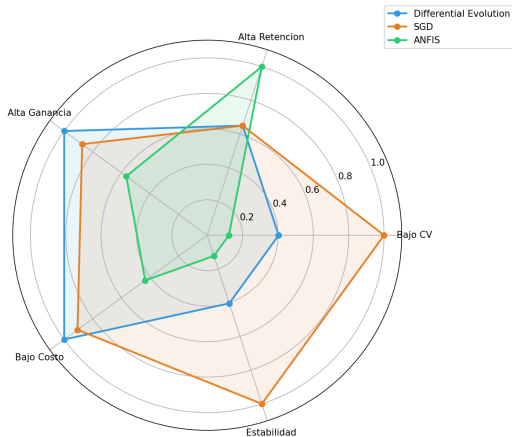
Métricas clave — barras comparativas

Comparacion de los 3 optimizadores — metricas clave



Perfil radial (spider chart)

Perfil comparativo de optimizadores



Cada eje: métrica normalizada al mejor valor.

- **DE:** domina en costo y ganancia
- **SGD:** mejor estabilidad (CV bajo)
- **ANFIS:** máxima retención

Área $\propto$ **desempeño global.**

Evolución temporal — trayectorias

Evolucion temporal — comparacion de optimizadores

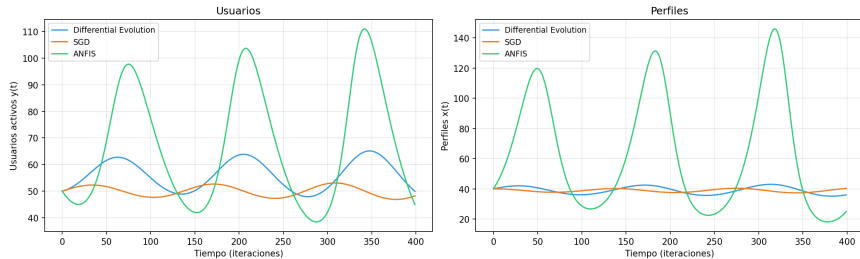
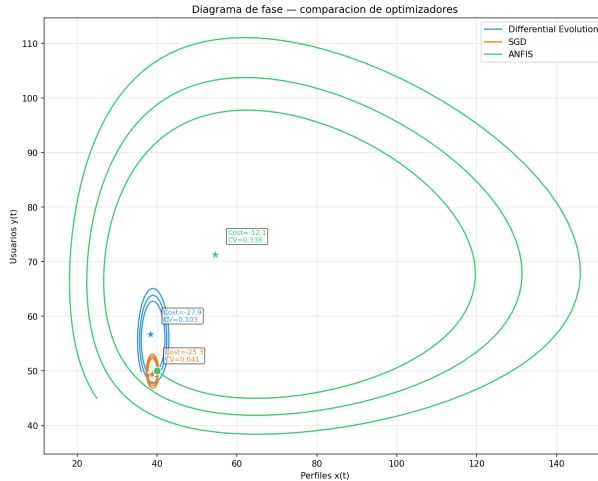


Diagrama de fase — comparación



Eficiencia: Costo vs Tiempo

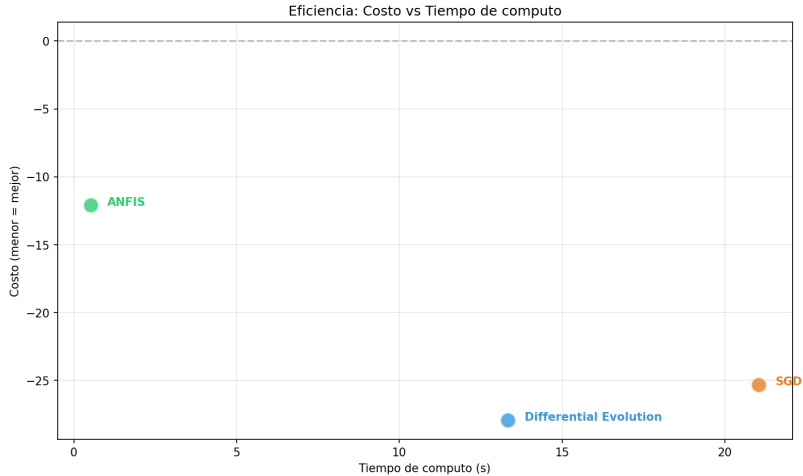


Tabla resumen comparativa

Optimizador	Costo	CV	Ret.	Gan.	Tiempo	Prec.
DE	−27.95	0.103	49.7%	\$70.60	13.0s	Alto
SGD	−25.34	0.041	49.7%	\$61.72	19.9s	Medio
ANFIS	−12.11	0.338	76.4%	\$39.95	0.5s	Bajo

Ranking final (puntuación 1-10)

#1 DE (6.66) #2 SGD (5.58) #3 ANFIS (4.60)

- 1 **DE es el mejor optimizador global:** encuentra el costo más bajo (-27.95) y la mayor ganancia ($\$70.60/\text{mes}$). Es la opción recomendada para producción.
- 2 **SGD es competitivo para re-optimización:** con grid de reinicios logra un costo cercano (-25.34) y la mejor estabilidad ($CV=0.041$), aunque es más lento.
- 3 **ANFIS ofrece un trade-off distinto:** maximiza retención (76.4%) sacrificando ganancia. Su tiempo de cómputo es mínimo ($0.5s$). Útil para explorar estrategias alternativas.
- 4 **El equilibrio $x^* = d/c$, $y^* = a/b$ se mantiene** como atractor del sistema independientemente del optimizador.

Recomendaciones estratégicas

Algoritmo de matching (c): dosificar coincidencias

Proporción x/y : ideal 0.5–1.0 perfiles por usuario

Óptimo: $a = 0.279$, $b = 0.005$, $c = 0.018$, $d = 0.700$

a b c d
crecimiento tasa de match eficiencia algoritmo abandono

Gracias

Preguntas?